

1. Project Title & Team

Title: Using the Lottery Ticket Hypothesis to Decrease the Size of LLMs

Name: [Len Washington III](#)

A#: A20484223

2. Problem & Motivation

For my semester project, I plan on using the Lottery Ticket Hypothesis (LTH)¹ to prune a Large Language Model to reduce the amount of space required to run said LLM. LLMs have become ingrained in most aspects of life since [Vaswani et al.](#) introduced the transformer architecture and ChatGPT made it easy to access them. However, the increasingly complex tasks that LLMs need to handle have led them to become larger and larger, which has increased the amount of space the LLMs need to be stored on and to run. Since the LTH requires for a model to have already been trained to competition before pruning, instead of trying to create a smaller model from the start, I thought this would be the best way to shrink pretrained models like Llama. The LTH is the way I plan to reduce the number of parameters for the transformer architecture, since some runs found that regular neural networks could have 5% of their original training size while still holding the same accuracy¹. To account for the difference in architectures between regular neural networks and transformers, I will consult the methods described by [Brix, Bahar, and Ney](#). The Lottery Ticket Hypothesis currently falls short due to it producing sparse matrices, which did not fare well with most hardware when the article was written.

3. Challenges & Trade-offs

The original challenges and trade-offs of the Lottery Ticket Hypothesis is that it would produce a sparse model. Back in 2018, the random sparsity of the model was an issue that most hardware architectures couldn't deal with, which may have made the sparse model run less efficiently than the full sized, dense model. To combat this, I'm going to use "structured sparsity", as proposed by [Ship X/Tech X](#), which would delete the weights in patterns instead of deleting weights randomly.

Another challenge is that [Frankle and Carbin](#) state that after pruning weights, the unpruned weights needed to be reset to the initial values that they had before training; however, it would be difficult to find a model that would have these initial values published. To combat this, I will attempt to pick new random values that may train a full transformer to reach similar accuracy levels as the pruned version. If I can't find values that work, I will note this in my paper and use either the values I downloaded, or a random initialization.

4. Proposed Design

1. **Initial Model Pruning** The first step involves downloading a small language model from HuggingFace with [SafeTensors](#) uploaded. This is done since I can directly ma-

nipulate and prune weights using SafeTensors.

2. **Generating the Pruning Algorithm for a SafeTensor file** The next step is to apply the algorithms in (1, 3) to the small model to reduce its size.
3. **Evaluate the test model** Each time I prune down the test model, I will evaluate its accuracy against a test dataset I download from HuggingFace. I currently plan on using the [WMT 2014 English-German](#) dataset, but I might talk to other students who are also modifying transformers to see which dataset they use.
4. **Apply these methods to Llama 4** If Steps 1–3 are successful, I will apply them to Llama 4 which has a model size of 67GB, and again evaluate this reduced Llama model in the same way as before.
5. **Apply to a domain-specific application** If Step 4 is successful (and I have the time), I might run Argonne National Laboratory’s [EXSCLAIM](#) pipeline using the modified model. As the current maintainer of this software, I would like to see how it fares taking the place of Llama 3.3 on our current hardware (that doesn’t have a GPU). I would record metrics and accuracy, and paste the responses in the appendix.

5. Evaluation Plan

Workloads

- [WMT 2014 English-German](#) (updated 2 years ago)

Environment

- Hardware
 - Personal PC
 - * Nvidia RTX 5090 GPU
 - * CPU Count: 32
 - * Ram: 64 GB
 - * Operating System: Ubuntu Linux
 - Chameleon?

- Pytorch
- SafeTensors

Metrics

- Number of Parameters
- Memory
- Accuracy

- Wait time
- Bilingual Evaluation Understudy [BLEU]³ (If the training dataset is similar to the one in (3) and translates between languages)

Baselines

- Table 1 of (3) describes baselines for the models they trained, including their sparsity memory.

Ablations

- Sparsity %

6. Plan & Milestones

Week 6-7 Figure out how to use SafeTensor and directly edit the smaller model.

Week 7-8 Run the LTH algorithm on the model, attempting to replicate the initial training values.

Week 9-11 Evaluate the algorithm will continuously decreasing the size of the model.

Week 12-13 If the model could be decreased, use the created files to run the same process for Llama 4.

Week 14-15 Results + artifacts ready.

7. References

- [1] Jonathan Frankle and Michael Carbin. “The Lottery Ticket Hypothesis: Training Pruned Neural Networks”. In: *CoRR* abs/1803.03635 (2018). arXiv: [1803.03635](https://arxiv.org/abs/1803.03635). URL: <https://arxiv.org/abs/1803.03635>.
- [2] Ashish Vaswani et al. “Attention is All you Need”. In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon et al. Vol. 30. Curran Associates, Inc., 2017. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- [3] Christopher Brix, Parnia Bahar, and Hermann Ney. “Successfully Applying the Stabilized Lottery Ticket Hypothesis to the Transformer Architecture”. In: *CoRR* abs/2005.03454 (2020). arXiv: [2005.03454](https://arxiv.org/abs/2005.03454). URL: <https://arxiv.org/abs/2005.03454>.
- [4] Ship X/Tech X. *MIT Proved You Can Delete 90% of a Neural Network*. Jan. 3, 2026. URL: <https://medium.com/techx-official/mit-proved-you-can-delete-90-of-a-neural-network-5c5f4aabf3b2>.